27

插叙：线程 API

本章简要介绍线程 API（thread API）的主要部分。在后面的章节里，我们会演示如何使用这些 API，届时还会进一步讲解每一部分。更多细节可以在各种书籍和网上资料中找到[B89, B97, B+96, K+96]。需要说明的是，后面几章会配合大量例子，放慢节奏地介绍锁（lock）和条件变量（condition variable）的概念；因此，本章更适合当作参考资料来用。

关键问题：如何创建和控制线程
操作系统应该为创建和控制线程（thread）提供哪些接口？这些接口应该怎样设计，才能既好用又实用？

27.1 创建线程

要编写多线程（multi-threaded）程序，首先得能创建新线程，所以必须有某种创建线程的接口。在 POSIX 中，这很简单：

```
#include <pthread.h>
int
pthread_create(pthread_t             *thread,
               const pthread_attr_t *attr,
               void                  *(*start_routine)(void*),
               void                  *arg);
```

这个声明看起来可能有点复杂（尤其是如果你没在 C 里用过函数指针的话），但其实没那么糟。它有四个参数：`thread`、`attr`、`start_routine` 和 `arg`。第一个参数 `thread` 是一个指向 `pthread_t` 类型结构的指针；我们要通过这个结构和该线程打交道，因此需要把它传给 `pthread_create()`，让它完成初始化。

第二个参数 `attr` 用来指定这个线程可能具有的属性，比如设置栈（stack）的大小，或者提供线程调度优先级（priority）方面的信息。属性要单独调用 `pthread_attr_init()` 来初始化，详见手册页（manual page）。不过在大多数情况下，用默认值就行；这时我们只需传入 `NULL`。

第三个参数最复杂，但它其实只是在问：这个线程应该从哪个函数开始运行？在 C 里，我们把它叫作**函数指针**（function pointer）。这个函数指针告诉我们，它需要的是这样一个

```
#include <stdio.h>
#include <pthread.h>

typedef struct {
    int a;
    int b;
} myarg_t;

void *mythread(void *arg) {
    myarg_t *args = (myarg_t *) arg;
    printf("%d %d\n", args->a, args->b);
    return NULL;
}

int main(int argc, char *argv[]) {
    pthread_t p;
    myarg_t args = { 10, 20 };

    int rc = pthread_create(&p, NULL, mythread, &args);
    ...
}
```

图 27.1：**创建线程**

函数：函数名为 `start_routine`，接受一个 `void *` 类型的参数（见 `start_routine` 后面括号里的内容），并返回一个 `void *` 类型的值（即一个 **void 指针**，void pointer）。

如果这个例程需要的是整数参数，而不是 void 指针，声明就会是这样：

```
int pthread_create(..., // 前两个参数不变
                   void *(*start_routine)(int),
                   int arg);
```

如果这个例程接受 void 指针作为参数，但返回一个整数，声明就会是这样：

```
int pthread_create(..., // 前两个参数不变
                   int (*start_routine)(void *),
                   void *arg);
```

最后，第四个参数 `arg` 就是要传给线程起始函数的那个参数。你可能会问：为什么要用这些 void 指针？答案很简单：把 void 指针作为 `start_routine` 函数的参数，我们就能传入任何类型的参数；把它作为返回值，线程就能返回任何类型的结果。

我们来看图 27.1 中的例子。这里只是创建了一个线程，给它传了两个参数，这两个参数打包成了我们自己定义的一个类型（`myarg_t`）。线程创建后，只需把参数强制转换成它期望的类型，就能按需取出这些参数。

就是这样！线程一旦创建，你就真正多了一个正在执行的实体，它有自己的调用栈（call stack），和程序中现有的所有线程运行在同一个地址空间（address space）里。好戏就此开场！

27.2 等待线程结束

上面的例子展示了如何创建线程。可是，如果想等某个线程结束，该怎么办呢？这得做点特别的事：调用例程 `pthread_join()`。

```
int pthread_join(pthread_t thread, void **value_ptr);
```

这个例程接受两个参数。第一个参数是 `pthread_t` 类型，用来指定要等待哪个线程。这个变量由线程创建例程初始化（你把指向它的指针作为参数传给 `pthread_create()` 时）；只要把它保留下来，就可以用它来等待那个线程结束。

第二个参数是一个指针，指向你期望拿回的返回值。因为这个例程可以返回任何东西，所以它被定义为返回一个 void 指针；又因为 `pthread_join()` 例程会修改传入参数的值，所以你需要传入指向这个值的指针，而不只是这个值本身。

我们再来看一个例子（图 27.2，第 3 页）。在这段代码中，同样创建了一个线程，并通过 `myarg_t` 结构给它传了几个参数。返回值用 `myret_t` 类型来传递。线程运行结束后，主线程（main thread）一直在 `pthread_join()` 例程[1]里等待，这时随即返回，我们就可以访问线程返回的值，也就是 `myret_t` 里的内容。

关于这个例子，有几点需要注意。首先，很多时候我们并不需要这样费劲地打包、拆包参数。例如，如果创建的线程不需要参数，创建时传入 `NULL` 作为参数就行。同样，如果不关心返回值，也可以给 `pthread_join()` 传入 `NULL`。

```
1  typedef struct { int a; int b; } myarg_t;
2  typedef struct { int x; int y; } myret_t;
3
4  void *mythread(void *arg) {
5      myret_t *rvals = Malloc(sizeof(myret_t));
6      rvals->x = 1;
7      rvals->y = 2;
8      return (void *) rvals;
9  }
10
11 int main(int argc, char *argv[]) {
12     pthread_t p;
13     myret_t *rvals;
14     myarg_t args = { 10, 20 };
15     Pthread_create(&p, NULL, mythread, &args);
16     Pthread_join(p, (void **) &rvals);
17     printf("returned %d %d\n", rvals->x, rvals->y);
18     free(rvals);
19     return 0;
20 }
```

图 27.2：**等待线程结束**

[1]注意，这里我们用的是包装函数；具体来说，我们调用的是 `Malloc()`、`Pthread_join()` 和 `Pthread_create()`，它们只是调用对应的小写版本，并确认这些例程没有返回什么意外的结果。

```
void *mythread(void *arg) {
    long long int value = (long long int) arg;
    printf("%lld\n", value);
    return (void *) (value + 1);
}

int main(int argc, char *argv[]) {
    pthread_t p;
    long long int rvalue;
    Pthread_create(&p, NULL, mythread, (void *) 100);
    Pthread_join(p, (void **) &rvalue);
    printf("returned %lld\n", rvalue);
    return 0;
}
```

图 27.3：**更简单的线程传参方式**

其次，如果只传一个值（例如一个 `long long int`），就不必把它打包成参数。图 27.3（第 4 页）给出了一个例子。这种情况下就简单一些，因为不必把参数和返回值装进结构体里。

第三，从线程返回值时一定要格外小心。具体来说，千万不要返回一个指针，让它指向分配在线程调用栈（call stack）上的东西。如果这样做，你觉得会发生什么？（好好想想！）下面是一段危险的代码，是从图 27.2 的例子改出来的。

```
1 void *mythread(void *arg) {
2     myarg_t *args = (myarg_t *) arg;
3     printf("%d %d\n", args->a, args->b);
4     myret_t oops; // 分配在栈上：错误！
5     oops.x = 1;
6     oops.y = 2;
7     return (void *) &oops;
8 }
```

在这个例子中，变量 `oops` 分配在 `mythread` 的栈上。然而，函数返回时，这个值会被自动释放（毕竟，这正是栈用起来这么方便的原因！）；因此，传回一个指向已释放变量的指针，会引发各种糟糕的后果。打印你以为已经返回的那些值时，你多半（但不一定！）会大吃一惊。自己试试看吧[2]！

最后，你可能会注意到，先用 `pthread_create()` 创建一个线程，紧接着就调用 `pthread_join()`，这种创建线程的方式相当奇怪。事实上，有一种更简单的办法能完成同样的事，它叫作**过程调用**（procedure call）。显然，我们通常会创建不止一个线程并等待它们结束，否则用线程就没多大意义了。

需要指出的是，并非所有多线程代码都会用到 join 例程。例如，一个多线程的 Web 服务器可能会创建若干工作线程（worker thread），然后用主线程接收请求，再把请求交给这些工作线程，如此一直运行下去。这类长期运行的程序可能就不需要 join。然而，如果一个

[2]幸运的是，你写出这样的代码时，编译器 `gcc` 很可能会发出警告，这又是一个要重视编译器警告的理由。

并行程序（parallel program）创建线程来（并行地）执行某项特定任务，它很可能会用 join 来确保所有这些工作都已完成，然后才退出或进入下一阶段的计算。

27.3 锁

除了创建线程和 join，POSIX 线程库（thread library）提供的下一组最有用的函数，大概就是用**锁**（lock）来为临界区（critical section）提供互斥（mutual exclusion）的那些函数了。为此要用到的最基本的一对例程是：

```
int pthread_mutex_lock(pthread_mutex_t *mutex);
int pthread_mutex_unlock(pthread_mutex_t *mutex);
```

这两个例程应该很容易理解和使用。如果有一段代码是**临界区**（critical section），因而需要保护起来以确保正确运行，锁就非常有用。你大概能想象出代码是什么样子：

```
pthread_mutex_t lock;
pthread_mutex_lock(&lock);
x = x + 1; // 或者你的临界区里要做的任何事
pthread_mutex_unlock(&lock);
```

这段代码的意图是这样的：如果调用 `pthread_mutex_lock()` 时没有其他线程持有锁，这个线程就会获取锁，进入临界区。如果确实有另一个线程持有锁，试图获取锁的线程就不会从这个调用中返回，直到它拿到锁为止（拿到锁，意味着原先持有锁的线程已经调用解锁（unlock），把锁释放了）。当然，在某一时刻，可能有许多线程都卡在获取锁的函数里等待；不过，只有已经拿到锁的线程才应该调用解锁。

遗憾的是，这段代码在两个重要的方面有问题。第一个问题是**缺少正确的初始化**（lack of proper initialization）。所有的锁都必须正确初始化，才能保证它们一开始就有正确的值，从而在加锁（locking）和解锁时按预期工作。

POSIX 线程（POSIX threads）提供了两种初始化锁的方法。第一种是像下面这样使用 `PTHREAD_MUTEX_INITIALIZER`：

```
pthread_mutex_t lock = PTHREAD_MUTEX_INITIALIZER;
```

这样会把锁设为默认值，从而让锁可用。动态的方法（即在运行时初始化）是调用 `pthread_mutex_init()`，如下所示：

```
int rc = pthread_mutex_init(&lock, NULL);
assert(rc == 0); // 一定要检查是否成功!
```

这个例程的第一个参数是锁本身的地址，第二个参数是一组可选的属性。属性的详细内容请自行阅读；传入 `NULL` 就表示使用默认值。两种方法都可以，不过我们通常用动态的（后一种）方法。注意，用完锁之后，还应该相应地调用 `pthread_mutex_destroy()`；所有细节请参阅手册页。

上面代码的第二个问题是，调用加锁和解锁时没有检查错误码。和你在 UNIX 系统中调用的几乎所有库例程一样，这些例程也可能失败！如果代码没有正确检查错误码，失败就会悄无声息地发生；在这个例子中，这可能让多个线程同时进入临界区。至少要使用包装函数，

```
// 让代码保持简洁；只有在失败时可以直接 exit() 才能这样用
void Pthread_mutex_lock(pthread_mutex_t *mutex) {
    int rc = pthread_mutex_lock(mutex);
    assert(rc == 0);
}
```

图 27.4：**一个包装函数的例子**

断言例程调用成功了，如图 27.4（第 6 页）所示；更复杂的（不是玩具的）程序出错时不能一退了之，应该检查调用是否失败，并在失败时做出适当的处理。

在 pthread 库中，和锁打交道的例程并不只有加锁和解锁。还有两个值得关注的例程：

```
int pthread_mutex_trylock(pthread_mutex_t *mutex);
int pthread_mutex_timedlock(pthread_mutex_t *mutex,
                            struct timespec *abs_timeout);
```

这两个调用都用来获取锁。如果锁已经被别的线程持有，`trylock` 版本会直接返回失败；`timedlock` 版本则会在超时（timeout）或者拿到锁之后返回，看哪个先发生。因此，超时时间为零的 `timedlock` 就退化成了 `trylock`。一般来说，这两个版本都应该避免使用；不过在少数情况下，避免卡在获取锁的例程里（也许是无限期地卡住）还是有用的，后面的章节里就会看到这样的例子（例如研究死锁（deadlock）的时候）。

27.4 条件变量

任何线程库的另一个重要组成部分（POSIX 线程当然也不例外）是**条件变量**（condition variable）。如果线程之间需要以某种方式互相发信号（signal），比如一个线程要等另一个线程做完某件事才能继续，条件变量就很有用。想以这种方式交互的程序，主要用到两个例程：

```
int pthread_cond_wait(pthread_cond_t *cond,
                      pthread_mutex_t *mutex);
int pthread_cond_signal(pthread_cond_t *cond);
```

要使用条件变量，还必须有一把和这个条件相关联的锁。调用上面任意一个例程时，都应该持有这把锁。

第一个例程 `pthread_cond_wait()` 让调用它的线程进入睡眠，等待其他线程给它发信号。发信号的时机通常是程序里发生了某种变化，而这个正在睡眠的线程可能关心这种变化。典型的用法是这样的：

```
pthread_mutex_t lock = PTHREAD_MUTEX_INITIALIZER;
pthread_cond_t  cond = PTHREAD_COND_INITIALIZER;

Pthread_mutex_lock(&lock);
while (ready == 0)
    Pthread_cond_wait(&cond, &lock);
Pthread_mutex_unlock(&lock);
```

在这段代码中，相关的锁和条件变量初始化之后[3]，线程检查变量 ready 是否已经被设成了非零值。如果没有，线程就调用等待例程进入睡眠，直到有别的线程把它唤醒。

唤醒线程的代码在另外某个线程里运行，大致是这样的：

```
Pthread_mutex_lock(&lock);
ready = 1;
Pthread_cond_signal(&cond);
Pthread_mutex_unlock(&lock);
```

关于这段代码，有几点需要注意。首先，发信号时（以及修改全局变量 ready 时），我们总是确保持有锁。这样可以保证我们不会不小心在代码里引入竞态条件（race condition）。

其次，你可能会注意到，等待调用的第二个参数是一把锁，而发信号的调用只接受一个条件变量。之所以有这个区别，是因为等待调用除了让调用线程进入睡眠，还会在让它睡眠的同时释放这把锁。想象一下，如果它不释放锁，另一个线程怎么可能拿到锁、再发信号唤醒它呢？不过，pthread_cond_wait() 在被唤醒后、返回之前，会重新获取这把锁，从而保证等待线程从等待序列开头获取锁、到结尾释放锁的这段时间里，只要它在运行，就一定持有锁。

最后还有一个怪异之处：等待线程是在一个 while 循环里重新检查条件，而不是用一个简单的 if 语句。在后面专门研究条件变量的章节里，我们会详细讨论这个问题；不过一般来说，用 while 循环是简单而安全的做法。虽然它会重新检查条件（也许会增加一点开销），但有些 pthread 实现可能会虚假地唤醒等待线程；在这种情况下，如果不重新检查，等待线程就会以为条件已经变了，而实际上并没有。因此，更稳妥的做法是把被唤醒看作一个提示，表示有些东西可能变了，而不是把它当作确凿的事实。

注意，有时你会想用一个简单的标志在两个线程之间传递信号，而不用条件变量和相关的锁。例如，我们可以把上面的等待代码改写成这样：

```
while (ready == 0)
    ; // 自旋
```

相应的发信号代码则是这样的：

```
ready = 1;
```

千万不要这样做，原因如下。首先，它在很多情况下性能很差：长时间自旋（spin）只会白白浪费 CPU 周期。其次，它容易出错。最近的研究表明 [X+10]，用标志（像上面那样）在线程之间同步，出错的可能性大得惊人；在那项研究中，这类 *ad hoc* 同步大约有一半都有 bug！不要偷懒；即使你觉得不用条件变量也能蒙混过关，也要用条件变量。

如果条件变量听起来让人困惑，（暂时）不用太担心，后面会有一章专门详细讲解。在那之前，只要知道有这么个东西，并大致了解怎样用、为什么要用就够了。

[3]也可以用 pthread_cond_init()（以及 pthread_cond_destroy()）代替静态初始化器 PTHREAD_COND_INITIALIZER。听起来更麻烦？确实更麻烦。

27.5 编译和运行

本章所有的代码示例都比较容易跑起来。要编译它们，代码里必须包含头文件 `pthread.h`。在链接命令中，还必须加上 `-pthread` 选项，显式地链接 pthreads 库。

例如，编译一个简单的多线程程序，只需这样做：

```
prompt> gcc -o main main.c -Wall -pthread
```

只要 `main.c` 包含了 pthreads 的头文件，你就成功编译了一个并发程序（concurrent program）。至于它能不能正常工作，和往常一样，那完全是另一回事了。

27.6 小结

我们介绍了 pthread 库的基础知识，包括创建线程、用锁实现互斥，以及用条件变量发信号和等待。要写出健壮、高效的多线程代码，除了耐心和万分小心，别的你也不需要多少了！

本章最后列出一组提示，在你编写多线程代码时可能会派上用场（详见下一页的补充）。API 还有其他一些有意思的方面；如果想了解更多，可以在 Linux 系统上输入 `man -k pthread`，查看构成整个接口的一百多个 API。不过，这里讨论的基础知识应该足以让你构建出复杂的（但愿也是正确且高性能的）多线程程序。线程的难点不在于 API，而在于构建并发程序时那些棘手的逻辑。请继续往下读，了解更多内容。

 © 2008–23, ARPACI-DUSSEAU

补充：使用线程 API 的指导原则

使用 POSIX 线程库构建多线程程序时（其实用任何线程库都一样），有一些虽小但很重要的事情需要记住。它们是：

- **保持简单**。最重要的是，在线程之间加锁或发信号的代码都应该尽可能简单。复杂的线程交互会导致 bug。
- **尽量减少线程之间的交互**。让线程之间交互的方式尽可能少。每一种交互都要仔细思考，并用经过验证的可靠方法来构建（其中很多方法我们会在后面几章学到）。
- **初始化锁和条件变量**。不这样做，代码就会时而正常、时而以非常奇怪的方式出错。
- **检查返回码**。当然，无论你写什么 C 和 UNIX 程序，都应该检查每一个返回码，这里也一样。不这样做，程序就会出现怪异而难以理解的行为，让你很可能（a）尖叫，（b）揪掉几根头发，或者（c）两者兼有。
- **注意给线程传参数、从线程返回值的方式**。尤其是，如果你传的是一个引用，指向栈上分配的变量，那多半就做错了。
- **每个线程都有自己的栈**。和上一点相关，请记住每个线程都有自己的栈。因此，如果线程正在执行的某个函数里有一个局部分配的变量，这个变量实际上是该线程私有的；其他线程无法（轻易）访问它。要在线程之间共享数据，这些值必须放在**堆**（heap）里，或者放在其他某个全局可访问的地方。
- **总是用条件变量在线程之间发信号**。虽然用一个简单的标志往往很诱人，但不要这样做。
- **查阅手册页**。尤其是在 Linux 上，pthread 的手册页信息非常丰富，讨论了这里提到的许多细微之处，而且往往讲得更详细。请仔细阅读！

参考文献

[B89] "An Introduction to Programming with Threads" by Andrew D. Birrell. DEC Technical Report, January, 1989. Available: https://birrell.org/andrew/papers/035-Threads.pdf *A classic but older introduction to threaded programming. Still a worthwhile read, and freely available.*

[B97] "Programming with POSIX Threads" by David R. Butenhof. Addison-Wesley, May 1997. *Another one of these books on threads.*

[B+96] "PThreads Programming: by A POSIX Standard for Better Multiprocessing." Dick Buttlar, Jacqueline Farrell, Bradford Nichols. O'Reilly, September 1996 *A reasonable book from the excellent, practical publishing house O'Reilly. Our bookshelves certainly contain a great deal of books from this company, including some excellent offerings on Perl, Python, and Javascript (particularly Crockford's "Javascript: The Good Parts".)*

[K+96] "Programming With Threads" by Steve Kleiman, Devang Shah, Bart Smaalders. Prentice Hall, January 1996. *Probably one of the better books in this space. Get it at your local library. Or steal it from your mother. More seriously, just ask your mother for it – she'll let you borrow it, don't worry.*

[X+10] "Ad Hoc Synchronization Considered Harmful" by Weiwei Xiong, Soyeon Park, Jiaqi Zhang, Yuanyuan Zhou, Zhiqiang Ma. OSDI 2010, Vancouver, Canada. *This paper shows how seemingly simple synchronization code can lead to a surprising number of bugs. Use condition variables and do the signaling correctly!*

 © 2008–23, ARPACI-DUSSEAU

作业（编程）

在这一节，我们会编写一些简单的多线程程序，并用一个叫作 **helgrind** 的专门工具来找出这些程序中的问题。如何构建这些程序、如何运行 helgrind，请阅读作业下载包里的 README。

问题

1. 首先构建 `main-race.c`。读一读这段代码，找出其中（希望很明显的）数据竞争（data race）。然后运行 helgrind（输入 `valgrind --tool=helgrind main-race`），看看它是怎样报告这个竞争的。它指出的代码行对吗？它还给了你哪些信息？
2. 把其中一行出问题的代码删掉，会发生什么？现在给对共享变量的其中一处更新加上锁，然后再给两处更新都加上锁。在这几种情况下，helgrind 分别报告了什么？
3. 现在来看 `main-deadlock.c`。读一读这段代码。它有一个叫作**死锁**（deadlock）的问题（后面会有一章更深入地讨论）。你能看出它可能有什么问题吗？
4. 现在对这段代码运行 helgrind。helgrind 报告了什么？
5. 现在用 helgrind 检查 `main-deadlock-global.c`。先读一读这段代码：它和 `main-deadlock.c` 有一样的问题吗？helgrind 应该报告同样的错误吗？这说明了 helgrind 这类工具的什么问题？
6. 接下来看 `main-signal.c`。这段代码用一个变量（`done`）来发信号，表示子线程（child thread）已经完成、父线程（parent thread）可以继续了。为什么这段代码效率很低？尤其是子线程要很久才能完成的时候，父线程最终把时间花在了什么上面？
7. 现在对这个程序运行 helgrind。它报告了什么？这段代码正确吗？
8. 现在看这段代码稍加修改后的版本，在 `main-signal-cv.c` 里。这个版本用条件变量和相关的锁来发信号。为什么这段代码比之前的版本更好？是因为正确性，还是性能，还是两者都有？
9. 再对 `main-signal-cv` 运行一次 helgrind。它报告了什么错误吗？

 © 2008–23, ARPACI-DUSSEAU